CSS3122 DATA STRUCTURE AND ALGORITHMS

CH4 POINTER

KHAIRUNNISA IBR

CHAPTER OUTLINE

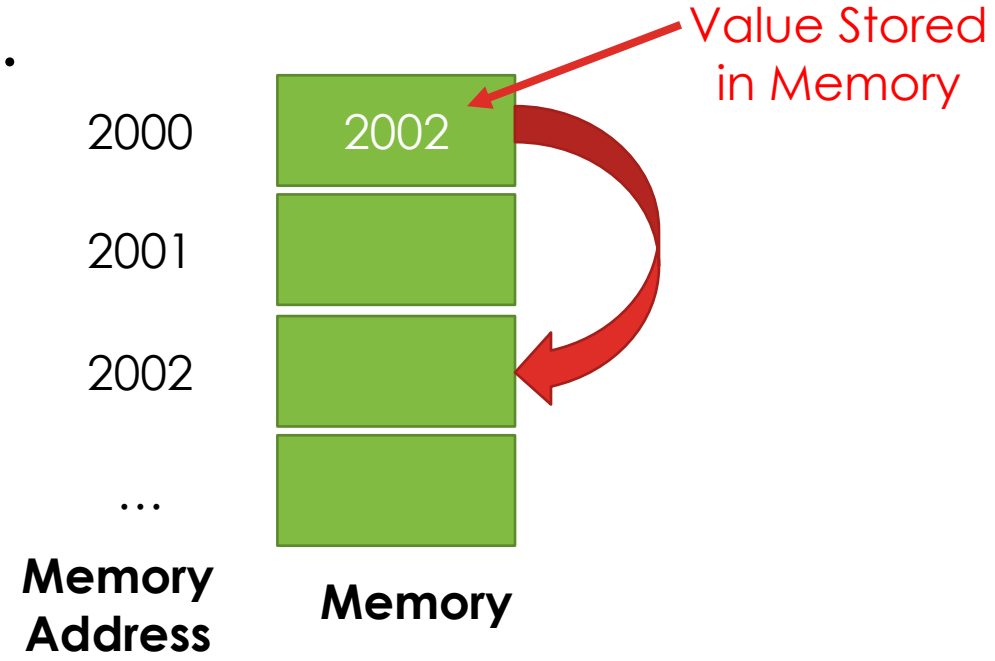
In this chapter, you will learn about:

1. Pointer basics
2. Pointer with functions
3. Call by reference
4. Array of pointers & pointer to array & programs

POINTER BASICS

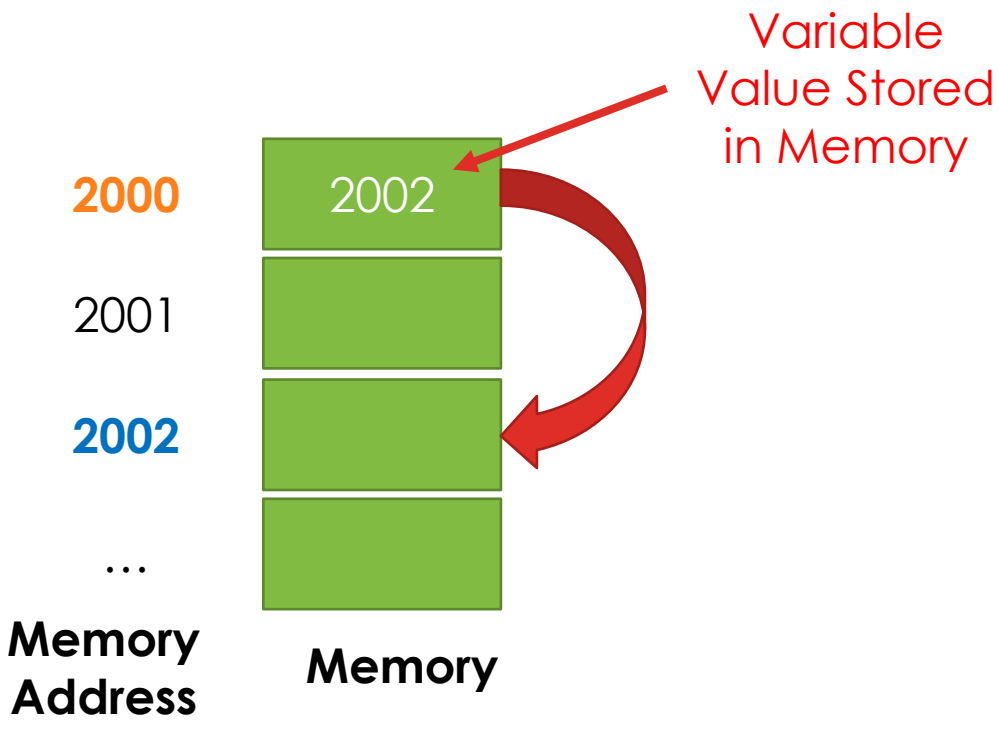
DEFINITION OF POINTER

- A derived data type that stores the address of another variable.
- Contains memory addresses as their values.
- Can be used to assign, access, and manipulate data values stored in the memory allocated to a variable since it can access the memory address of that variable.



POINTER BASICS

CONCEPT OF POINTER



- Memory address = address of variables in the memory.
- Value stored at memory address **2002** can be accessed by using address stored as value at memory address **2000**.
- It can be said that memory address **2000** points to the variable stored in memory address **2002**.
- Variable in memory address **2000** is a pointer variable.

POINTER BASICS

POINTER DECLARATION

- The general syntax of declaring pointer variables:

```
dataType *ptrName;
```

- Here, the data type is the data type of the value that the pointer will point to.

- For example:

```
int *pnum; //to point to integer variable
char *pch; //to point to char variable
float *pfnum; //to point to float variable
```

POINTER BASICS (CONT…)

EXAMPLE 1:

- The general syntax of assigning pointer values:

```
int x=10;
int *ptr;
ptr = &x;
```

- ptr is the name of the pointer variable.
- The * informs the compiler that ptr is a pointer variable.
- The int specifies that it will store the address of an integer variable.
- Hence, here an integer pointer variable points to an integer variable.

POINTER BASICS (CONT…)

USE OF & AND *

- **When is & used?**
- **When is * used?**

- & - "address operator" which gives or produces the memory address of a data variable
- * - "dereferencing operator" which provides the contents in the memory location specified by a pointer

POINTER BASICS (CONT…)

EXAMPLE 1:

```
int x=10;
int *ptr;
ptr = &x;
```

			10		
1000	1001	1002	1003	1004	1005

Memory Representation

- If X is assigned memory locations 1003 and 1004, hence &x is equal to 1003 (starting address of x),

  i.e. if **ptr = &x,** then **ptr = 1003.**
- We use unary * operator to refer to the variable value pointed by the pointer.
- E.g. ***ptr = 10.**

POINTER BASICS (CONT…)

EXAMPLE 2:

```
#include <stdio.h>
int main()
{
    int x;
    int *ptr;
    x = 10;
    ptr = &x;
    *ptr = *ptr + 1;

    printf("x = %d\n", x);
    printf("&x = %d\n", &x);
    printf("ptr = %d\n", ptr);
    printf("*ptr = %d\n", *ptr);
    printf("&*ptr = %d\n", &*ptr);

    return 0;
}
```

```
x = 11
&x = 1245024
ptr = 1245024
*ptr = 11
&*ptr = 1245024
Press any key to continue . .
```

POINTER BASICS (CONT…)

EXAMPLE 3:

```
#include<stdio.h>
int main()
{
        int x=2,y;
        int *ptr;
        ptr = &x;
        y = *ptr;
        printf("Value stored in x is %d\n",x); //1
        printf("Address of %d is %u\n",x,&x); //2
        printf("Address of %d is %u\n",*&x,&x); //3
        printf("Address of %d is %u\n",*ptr,ptr); //4
        printf("Address %d is stored at %u\n",ptr,&ptr); //5
        printf("Address of %d is %u\n",y,&y); //6
        *ptr = 25;
        printf("Now the value stored in x is %d\n",x); //7
        return 0;
}
```

Output?

```
1    Value stored in x is 2
2    Address of 2 is 890966264
3    Address of 2 is 890966264
4    Address of 2 is 890966264
5    Address 890966264 is stored at 890966272
6    Address of 2 is 890966268
7    Now the value stored in x is 25
```

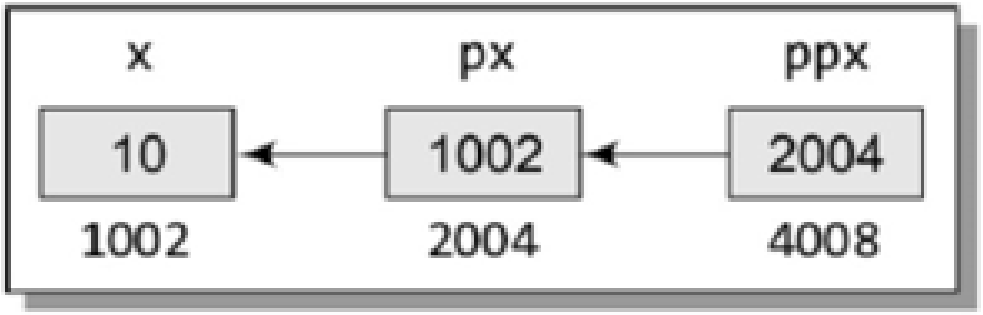
POINTER BASICS (CONT…)

POINTER TO A POINTER

- Consider the following code:

```
int x=10;
int *px, **ppx;
px = &x;
ppx = &px;
```

- The memory locations of the variables are as follows:




```
printf("%d",**ppx);
```

would output 10, i.e. the value of x.

POINTER WITH FUNCTIONS

- A function can have pointers as its parameters.
- E.g. **int functName(int *, int *);**

- A function can also return multiple values through pointer parameters.
- E.g. **int *functName(int *, int *);**

POINTER WITH FUNCTIONS (CONT…)

POINTERS AS FUNCTION PARAMETERS

- Write a program to add 2 integers using pointers and functions.

```
#include <stdio.h>
void sum (int*, int*, int*);
int main()
{
        int num1, num2, total;
        printf("\n Enter the first number : ");
        scanf("%d", &num1);
        printf("\n Enter the second number : ");
        scanf("%d", &num2);
        sum(&num1, &num2, &total);
        printf("\n Total = %d", total);
        return 0;
}
void sum (int *a, int *b, int *t)
{
        *t = *a + *b;
}
```

Output:

```
Enter the first number : 23
Enter the second number : 34
Total = 57
```

POINTER WITH FUNCTIONS (CONT…)

RETURNING POINTERS FROM FUNCTIONS

- Write a program to find a larger number between 2 numbers.

```
#include<stdio.h>
int *Large(int *, int *);
int main()
{
	int a=5,b=10,*p;
	p = Large(&a,&b);
	printf("Larger number between %d and %d is %d.",a,b,*p);
}
int *Large(int *x, int *y)
{
	if(*x>*y)
	return (x);
	else
	return (y);
}
```

Output:

```
Larger number between 5 and
10 is 10.
```

CALL BY REFERENCE

- In calling a function, we can use 2 methods to pass the values of a variable to a function definition from the function call.
  1. Call by reference.
  2. Call by value.

Call by reference

- The process of **passing the addresses of variables** to a function using **pointers**

Call by value

- The process of **passing the actual values of variables** to a function

CALL BY REFERENCE (CONT…)

- The function which is called by reference can modify the values of variables used in the call.
- This method is also known as **call by address** or **pass by pointers**.

CALL BY REFERENCE (CONT…)

Example:

- Shows how the addresses of variables are passed to a function using pointers.
- To change the values of these variables stored in two different locations in the memory.

```
#include <stdio.h>
void swap(int * , int *);
int main()
{
        int x = 2;
        int y = 4;
        printf("x is %d and y is %d before swapping\n",x,y);
        swap(&x,&y);
        printf("x is %d and y is %d after swapping\n",x,y);
        return 0;
}
void swap (int *x, int *y)
{
        int temp;
        temp = *x;
        *x=*y;
        *y=temp;
}
```

OUTPUT:

x is 2 and y is 4 before swapping
x is 4 and y is 2 after swapping

ARRAY OF POINTERS & POINTER TO ARRAY & PROGRAMS

ARRAY OF POINTERS

- An array of pointers can be declared as:

```
int *ptr[10]; //declares an array of 10 pointers, each pointer points
              //to an integer
```

- **Example:**

```
int *ptr[10];
int p=1, q=2, r=3, s=4, t=5;
ptr[0]; = &p;
ptr[1] = &q;
ptr[2] = &r;
ptr[3] = &s;
ptr[4] = &t;
printf("%d", *ptr[3]); //what is the output?
```

Output:
4

Because ptr[3] stores the address of integer variable s.

ARRAY OF POINTERS & POINTER TO ARRAY & PROGRAMS (CONT…)

- Another example:

```
int main()
{
   int arr1[] = {1,2,3,4,5};
   int arr2[] = {0,2,4,6,8};
   int arr3[] = {1,3,5,7,9};
   int *parr[3] = {arr1, arr2, arr3};
   int i;
   for (i=0; i<3; i++)
      printf("%d", *parr[i]);
   return 0;
}
```

Output:
1 0 1

Because parr[0] stores the base address of arr1, i.e. &arr1[0].
Thus, *parr[0]=1 (value pointed to by &arr1[0]).
Same case for *parr[1] and *parr[2].

ARRAY OF POINTERS & POINTER TO ARRAY & PROGRAMS (CONT…)

POINTER TO ARRAY & PROGRAMS

- Array name is a pointer to the first element.
  - Address of 1st array element is &x[0] or x.
  - Address of 2nd array element is &x[1] or (x+1).
  - So, address for ith element is &x[i] or (x+i).
- Then to retrieve the content of i element is x[i] or *(x+i).

ARRAY OF POINTERS & POINTER TO ARRAY & PROGRAMS (CONT…)

- Take note that:

arr[i] is equal to ***(arr+i)**

pv[i] is equal to ***(pv+i)**

Array notation

Pointer notation

ARRAY OF POINTERS & POINTER TO ARRAY & PROGRAMS (CONT…)

- Example:

```
#include <stdio.h>
int main()
{
    static int x[10] = {10,11,12,13,14,15,16,17,18,19};

    int i;
    for (i = 0; i <= 9; ++i)
    {
        printf("\n i = %d\tx[i] = %d\t*(x+i) = %d", i, x[i], *(x+i));
        printf("\t &x[i] = %X\t x+i = %X\n", &x[i], (x+i));
    }
        return 0;
}
```

```
i = 0  x[i] = 10      *(x+i) = 10      &x[i] = 417054  x+i = 417054

i = 1  x[i] = 11      *(x+i) = 11      &x[i] = 417058  x+i = 417058

i = 2  x[i] = 12      *(x+i) = 12      &x[i] = 41705C  x+i = 41705C

i = 3  x[i] = 13      *(x+i) = 13      &x[i] = 417060  x+i = 417060

i = 4  x[i] = 14      *(x+i) = 14      &x[i] = 417064  x+i = 417064

i = 5  x[i] = 15      *(x+i) = 15      &x[i] = 417068  x+i = 417068

i = 6  x[i] = 16      *(x+i) = 16      &x[i] = 41706C  x+i = 41706C

i = 7  x[i] = 17      *(x+i) = 17      &x[i] = 417070  x+i = 417070

i = 8  x[i] = 18      *(x+i) = 18      &x[i] = 417074  x+i = 417074

i = 9  x[i] = 19      *(x+i) = 19      &x[i] = 417078  x+i = 417078
Press any key to continue . . .
```

ARRAY OF POINTERS & POINTER TO ARRAY & PROGRAMS (CONT…)

- Since one dimensional array can be represented by pointer (the array name),
- Pointer can also used in representing multidimensional array.
- A multidimensional array can be declared as below:

Datatype (*ptr)[*expression1*] [*expression2*]…[*expression i*]

- Noted that for

int x[ ][20];

- It becomes

int (*x)[20];

ARRAY OF POINTERS & POINTER TO ARRAY & PROGRAMS (CONT…)

- To retrieve content of one dimensional array for array declared using pointer, it will be

***(arrayname + i)**

Example:

- to retrieve x[2][5] in array declared using pointer, the code will be

***(*(x+2)+5)**

ARRAY OF POINTERS & POINTER TO ARRAY & PROGRAMS (CONT…)

- Since array name is the pointer to the first element,
- When dealing with strings, the retrieval of the strings can be done by calling the content pointer to the array

*** arrayname**

ARRAY OF POINTERS & POINTER TO ARRAY & PROGRAMS (CONT…)

- **EXAMPLE:**

```
//using array
#include <stdio.h>
char x[] = "Externally";
int main()
{
   static char y[ ] = "Internally";

   printf("\n%s", x);
   printf("\n%s\n", y);
   return 0;
}
```

```
Externally
Internally
Press any key to continue . . .
```

```
//using pointer
#include <stdio.h>
char *x = "Externally";
int main()
{
   static char *y =
   "Internally";

   printf("\n%s", x);
   printf("\n%s\n", y);
   return 0;
}
```

```
Externally
Internally
Press any key to continue . . .
```